

CE802 2016 - Worked Solutions

Machine Learning and Data Mining

Full explanation set with formulas, derivations, and brief supporting illustrations

This document solves the full uploaded 2016 CE802 paper question by question. For numerical parts, the formulas are shown first and the arithmetic is then carried through step by step. For discursive parts, the answer is organised as clear marking points rather than a bare final statement.

Question 1 - Reinforcement Learning / MDPs

(a) What a Markov Decision Process is

A **Markov Decision Process (MDP)** is a mathematical model for sequential decision making under uncertainty. At each time step, an agent observes the current state, chooses an action, receives a reward, and moves to a new state. The word *Markov* means that the next state depends only on the *current* state and action - not on the full past history.

A standard finite MDP is written as $\langle S, A, T, R, \gamma \rangle$:

Symbol	Meaning
S	The set of states. Each state summarises the situation the agent can be in.
A	The set of actions. In many texts this is written as $A(s)$ when the available actions depend on the current state.
$T(s' s,a)$	The transition model: the probability of moving to next state s' after taking action a in state s .
$R(s,a)$ or $R(s,a,s')$	The immediate reward obtained after acting. This measures how desirable a transition is.
γ	The discount factor, $0 \leq \gamma \leq 1$, controlling how much future rewards matter relative to immediate reward.

The agent's objective is usually to maximise the expected discounted sum of rewards over time. This is why MDPs are the basic formal model behind reinforcement learning.

(b) Meaning of the Bellman optimality equation

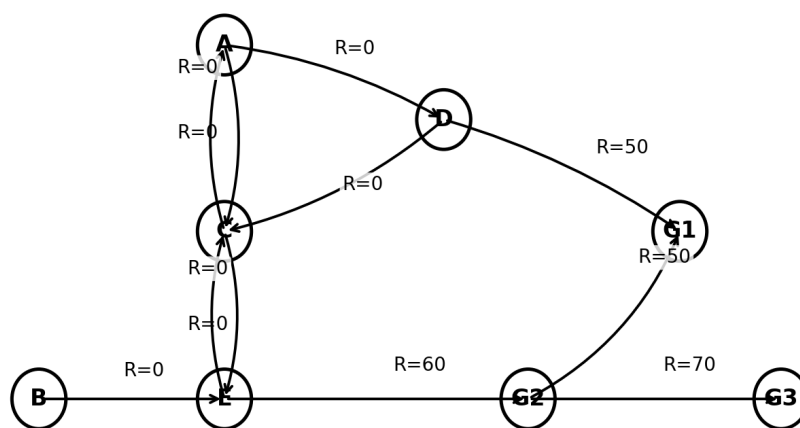
The equation in the question is the Bellman optimality equation for deterministic transitions:

$$V^*(s) = \max_a [R(s,a) + \gamma V^*(s')]$$

It says that the optimal value of a state s is found by examining *all* actions available in s , adding together the immediate reward from that action and the discounted value of the next state reached, and then choosing the action that gives the maximum total. The equation therefore expresses the principle of *optimal substructure*: an optimal policy from the current state must lead into an optimal policy thereafter.

(c) Discounted cumulative value of every state for $\gamma = 0.5$

Because the transitions are deterministic, the cleanest method is **backward evaluation** from the goal states. The transition structure reconstructed from the figure is shown below.



Recreated state-transition diagram used for the calculations.

Terminal states G1 and G3 have no outgoing reward-bearing transitions shown, so their future value is 0 after arrival:

$$V(G1) = 0, V(G3) = 0$$

Now evaluate the remaining states from right to left.

State G2 has two choices: to G1 with reward 50, or to G3 with reward 70.

$$V(G2) = \max(50 + 0.5 \times V(G1), 70 + 0.5 \times V(G3)) = \max(50, 70) = 70.$$

State E goes deterministically to G2 with reward 60.

$$V(E) = 60 + 0.5 \times V(G2) = 60 + 35 = 95.$$

State B goes deterministically to E with reward 0.

$$V(B) = 0 + 0.5 \times V(E) = 47.5.$$

For C, D and A there are competing actions, so write the maximisation explicitly:

$$V(C) = \max(0 + 0.5 \times V(A), 0 + 0.5 \times V(E)) = \max(0.5V(A), 47.5)$$

$$V(D) = \max(0 + 0.5 \times V(C), 50 + 0.5 \times V(G1)) = \max(0.5V(C), 50)$$

$$V(A) = \max(0 + 0.5 \times V(C), 0 + 0.5 \times V(D))$$

Since the path C → E already gives 47.5, D's best choice is the direct transition to G1 because 50 is larger than $0.5 \times 47.5 = 23.75$. Hence $V(D)=50$. Once $V(D)=50$, state A prefers A → D because $0.5 \times 50 = 25$ is larger than $0.5 \times 47.5 = 23.75$. Therefore $V(A)=25$. Substituting back confirms that C indeed prefers C → E, so $V(C)=47.5$.

State	Optimal value	Best action / reason
G1	0	Terminal
G3	0	Terminal
G2	70	Choose G2 → G3 because $70 > 50$
E	95	E → G2, so $60 + 0.5 \times 70$
B	47.5	B → E, so $0 + 0.5 \times 95$
D	50	Choose D → G1 because $50 > 0.5 \times 47.5$
A	25	Choose A → D because $0.5 \times 50 > 0.5 \times 47.5$

C	47.5	Choose C -> E because $0.5 \times 95 > 0.5 \times 25$
----------	------	---

Hence the discounted cumulative state values are: $V(A)=25$, $V(B)=47.5$, $V(C)=47.5$, $V(D)=50$, $V(E)=95$, $V(G1)=0$, $V(G2)=70$, $V(G3)=0$.

(d) Optimal action from the learned Q-values

Q-learning chooses the action with the **largest** Q-value, because larger Q means larger expected discounted return. Even when all three values are negative, the best action is the *least negative* one:

$$Q(s_0, a_0) = -6, Q(s_0, a_1) = -3, Q(s_0, a_2) = -1$$

Therefore the optimal action is **a2**, since $-1 > -3 > -6$.

Question 2 - Instance-Based Learning

(a) Meaning and essential components

Instance-based learning stores examples and postpones most of the generalisation work until a new query arrives. Instead of fitting one explicit global model in advance, the learner compares a new case against retained cases and predicts from the most similar ones. This is why it is often described as *lazy learning*.

A practical instance-based system needs the following core components:

- A representation of each example as a vector or structured record of attribute values.
- A similarity or distance measure (for example Euclidean, Manhattan, overlap, or a mixed-type distance).
- A storage policy deciding which training instances are retained.
- A retrieval rule specifying how neighbours are found (for example 1-NN or k-NN).
- A prediction rule, such as majority vote for classification or averaging for regression.
- Optional normalisation / weighting so that one large-scale attribute does not dominate the distance.

(b) Why retaining every training example is undesirable

Memory cost: storing all examples can become expensive when the data set is large.

Prediction time: each query may need to compare against every stored example, which makes classification slow.

Noise sensitivity: unusual or mislabeled points remain in the memory and can distort local decisions.

Redundancy: many interior points add little information because nearby examples of the same class already support the same decision.

(c) A successful retention method: condensed nearest neighbour (CNN / IB2-style editing)

A standard and successful way to reduce memory is to keep only the training examples that are needed to preserve the decision boundary.

One simple condensed procedure is:

- 1 Start the retained set with one seed example (or one seed per class).
- 2 Scan the remaining training examples in sequence.
- 3 Classify each example using 1-nearest neighbour on the currently retained set.
- 4 If the example is classified correctly, discard it as redundant.
- 5 If it is misclassified, add it to the retained set.
- 6 Repeat passes until no further additions are required.

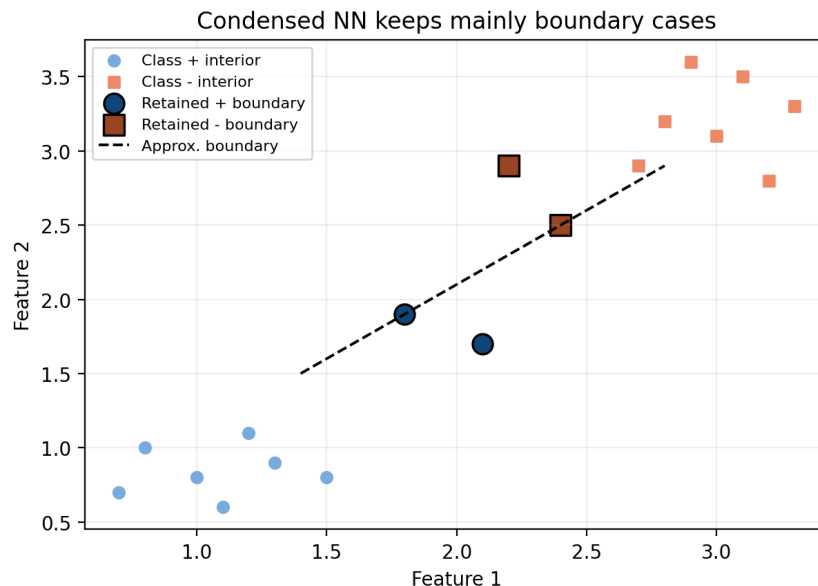


Illustration: dark points are the retained boundary cases; lighter interior points are usually unnecessary.

This works because **interior** examples are normally surrounded by points of the same class and are therefore already classified correctly by nearby retained prototypes. Examples near the class boundary, however, are exactly the ones that determine where the decision surface should lie, so these are the cases that tend to be kept.

A limitation is that noisy boundary points may also be retained, so in practice condensed storage is often combined with later editing or noise filtering.

(d) Difficulties caused by age, income, occupation and telephone number in a k-NN system

Attribute	Problem for k-NN	Reasonable treatment
Age	Numeric, but on a different scale from some other attributes	Normalise or standardise so age contributes proportionately to distance
Income	Large numeric scale may dominate Euclidean distance	Rescale (for example z-score or min-max normalisation); possibly use a non-Euclidean distance
Occupation	Nominal category: codes such as 1,2,3 do not imply any meaningful ordering	Use meaningful encoding and a discrete-type similarity measure rather than distance
Telephone number	Essentially an identifier, not a meaningful behavioural descriptor	Remove; almost entirely because it adds noise and spurious information

Overall, the key idea is that k-NN depends critically on the *meaningfulness of the distance function*. Good preprocessing - scaling numeric features, handling categorical variables correctly, and removing identifiers - is therefore essential.

Question 3 - Evaluation and Classifier Cooperation

(a) Why 95% accuracy alone is not enough

An accuracy figure can only be interpreted properly when it is placed in context. At least two of the following are needed before deciding whether 95% is good:

- **Class distribution / baseline accuracy.** If 95% of the data belong to one class, then always predicting that class already gives 95%, so the reported result would not be impressive.
- **Confusion matrix or class-specific metrics.** We need recall, precision, specificity, or F1 because a system may achieve high overall accuracy while performing badly on the minority class.
- **Error costs.** Some mistakes are far more serious than others (for example missing a disease versus a false alarm), so equal-weight accuracy may hide an unacceptable model.
- **Evaluation protocol.** Was the score measured on independent test data, through cross-validation, or on the training data itself? A training-set score may overestimate future performance.

(b) Performance measures from the confusion matrix

From the table in the question:

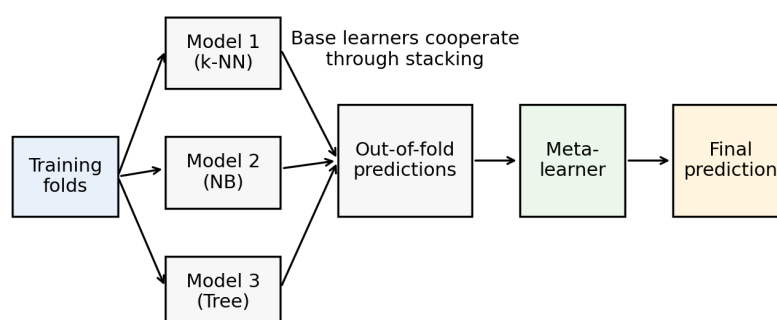
	Predicted Yes	Predicted No
Actual Yes	39	11
Actual No	9	74

Therefore: TP = 39, FN = 11, FP = 9, TN = 74, and the total number of cases is $N = 39 + 11 + 9 + 74 = 133$.

Measure	Formula	Substitution and value
Estimated prior of class Yes	$(TP + FN) / N$	$(39 + 11) / 133 = 50 / 133 = 0.3759$
Recall / sensitivity	$TP / (TP + FN)$	$39 / 50 = 0.7800$
Precision	$TP / (TP + FP)$	$39 / 48 = 0.8125$
Classification accuracy	$(TP + TN) / N$	$(39 + 74) / 133 = 113 / 133 = 0.8496$

(c) A cooperative learning method: stacked generalisation (stacking)

When several learning procedures have each achieved partial success, a strong way to make them cooperate is to build an **ensemble**. For genuinely different model families, the most flexible approach is **stacking**.



Stacking lets heterogeneous learners contribute complementary strengths.

A detailed stacking procedure is:

- 1 Train several different base learners on the same training problem - for example k-NN, Naive Bayes, and a decision tree.
- 2 Use cross-validation or a held-out validation scheme to obtain out-of-fold predictions from each base learner on examples it did not train on.
- 3 For each example, collect the base learners' predictions (or probabilities) into a new feature vector.
- 4 Train a meta-learner on these new features. The meta-learner learns when each base model is reliable and how to combine them.
- 5 At test time, each base learner predicts first; their outputs are then fed to the meta-learner, which produces the final prediction.

Why this can help: different algorithms tend to make different errors. A distance-based learner may work well in one region of the space, while a probabilistic learner or tree may work better elsewhere. If their errors are not perfectly correlated, the combined system can reduce variance and improve accuracy beyond any single learner on its own.

A simpler alternative is majority voting, but stacking is usually stronger because the combination itself is learned from data rather than fixed by hand.